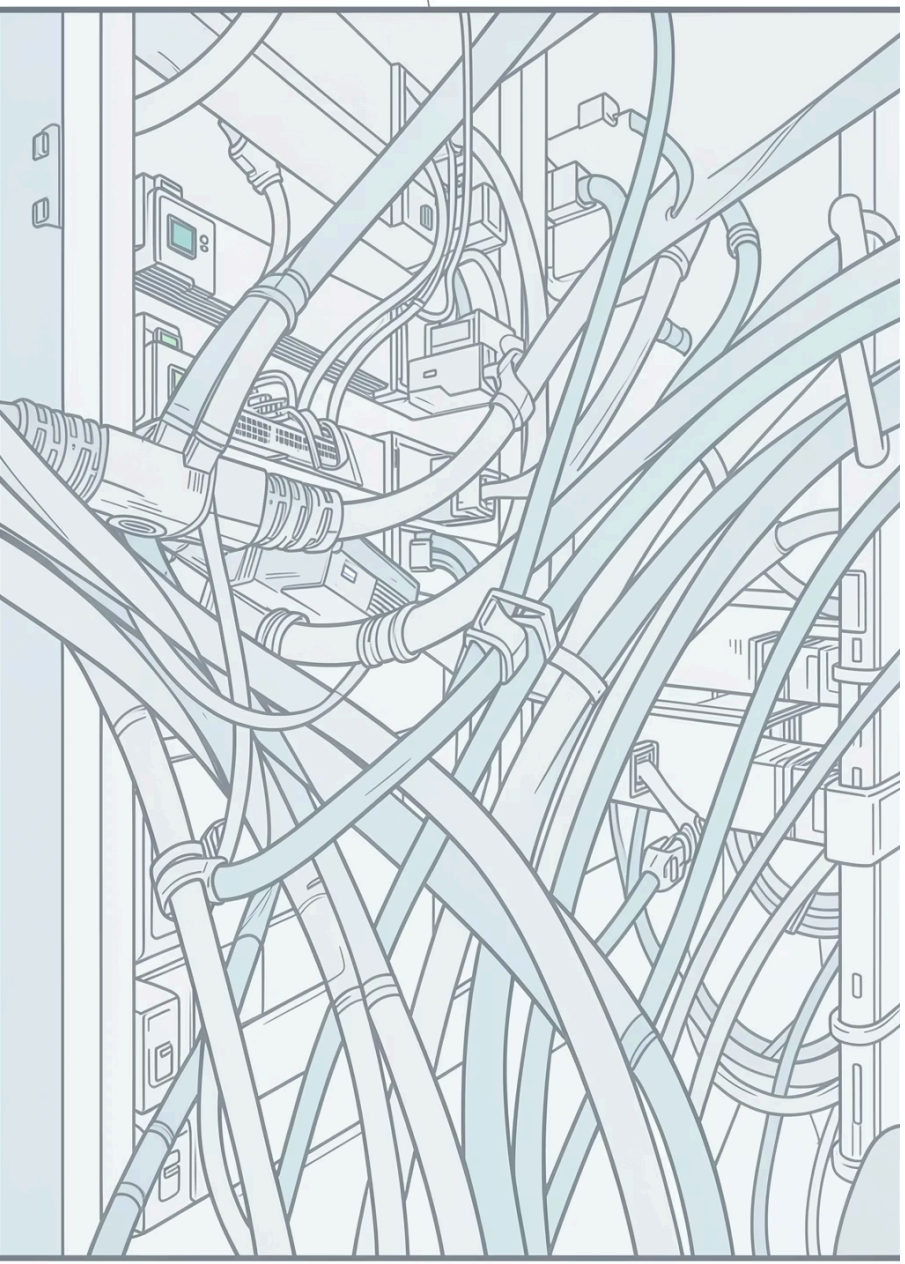
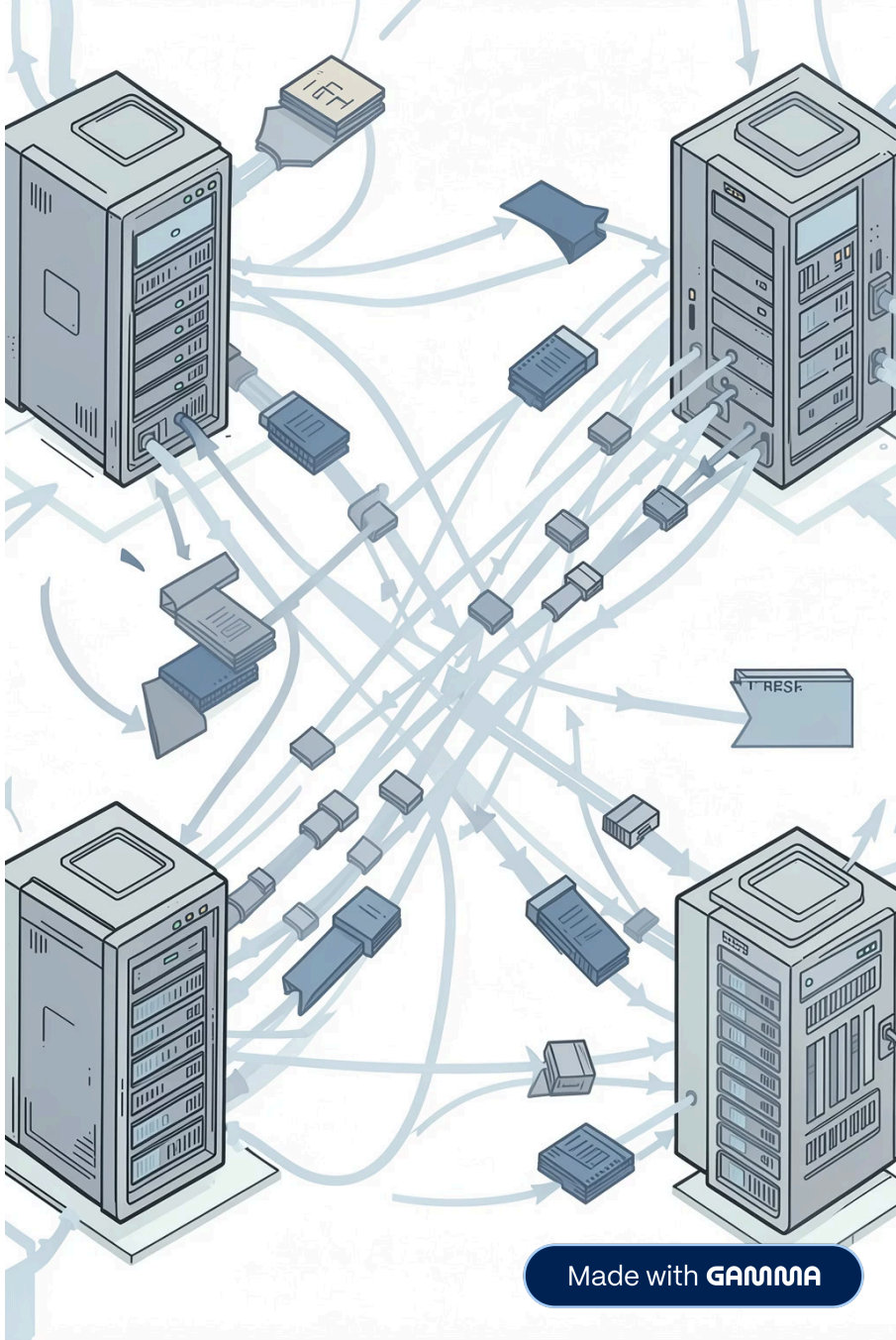


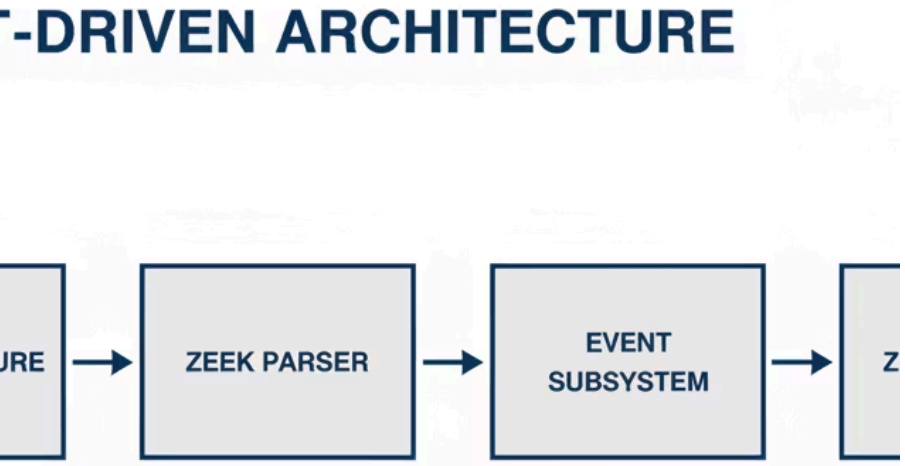
Zeek: Passive IDS and Attack Simulation

Names: Marco Pino, Matias Reyes, Joaquin Carranza, Juan Andino

Introduction to Zeek

- Zeek is an event-oriented network traffic analysis framework.
- It works as a *passive* IDS: it monitors traffic, generates logs and alerts, and does not block by default.
- Strong focus on visibility and forensic analysis: building rich records (conn.log, notice.log, weird.log).





How Does Zeek Work?

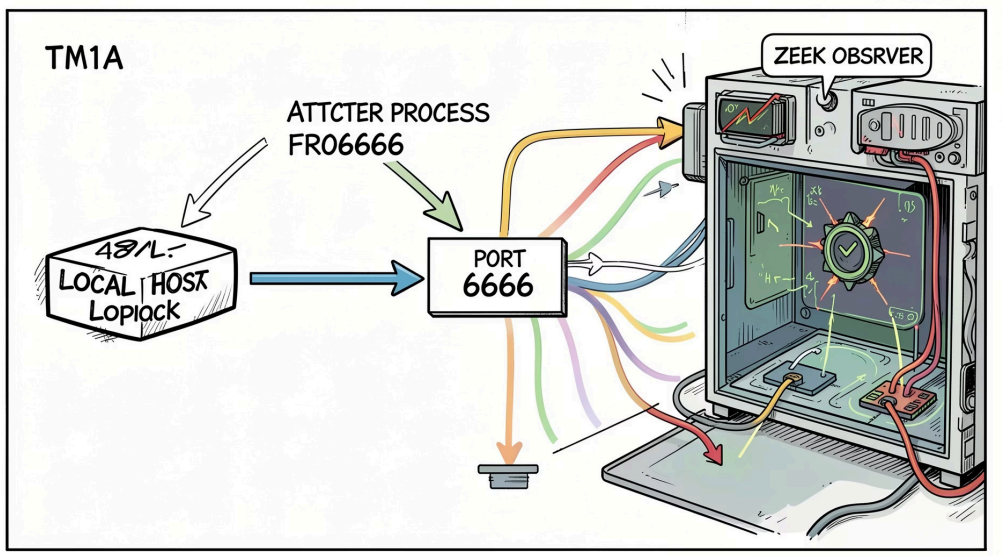
- Captures packets from an interface (e.g., loopback 'lo' / 127.0.0.1) using libpcap.
- Triggers a chain of events (TCP connection, HTTP request, etc.) in real time.
- Scripts written in the Zeek language process those events and generate logs/notices according to the defined logic.

The diagram summarizes the flow: packet → parsing → event → script → log/alert.

The laboratory: Project architecture

Scenario:

- Simulated local network using loopback 127.0.0.1.
- Attacker: machine running attack.py (TCP connections to port 6666).
- Defender: Zeek sensor in monitor mode observing traffic on 'lo'.
- Objective: demonstrate detection of connection attempts to a forbidden port.



list.zeek — the blacklist

Purpose: define a reusable set of suspicious ports for other scripts.

```
# Illustrative snippet (list.zeek)
const suspicious_ports: set[port] = {
    6666/tcp,
    12345/tcp
};
```

Notes:

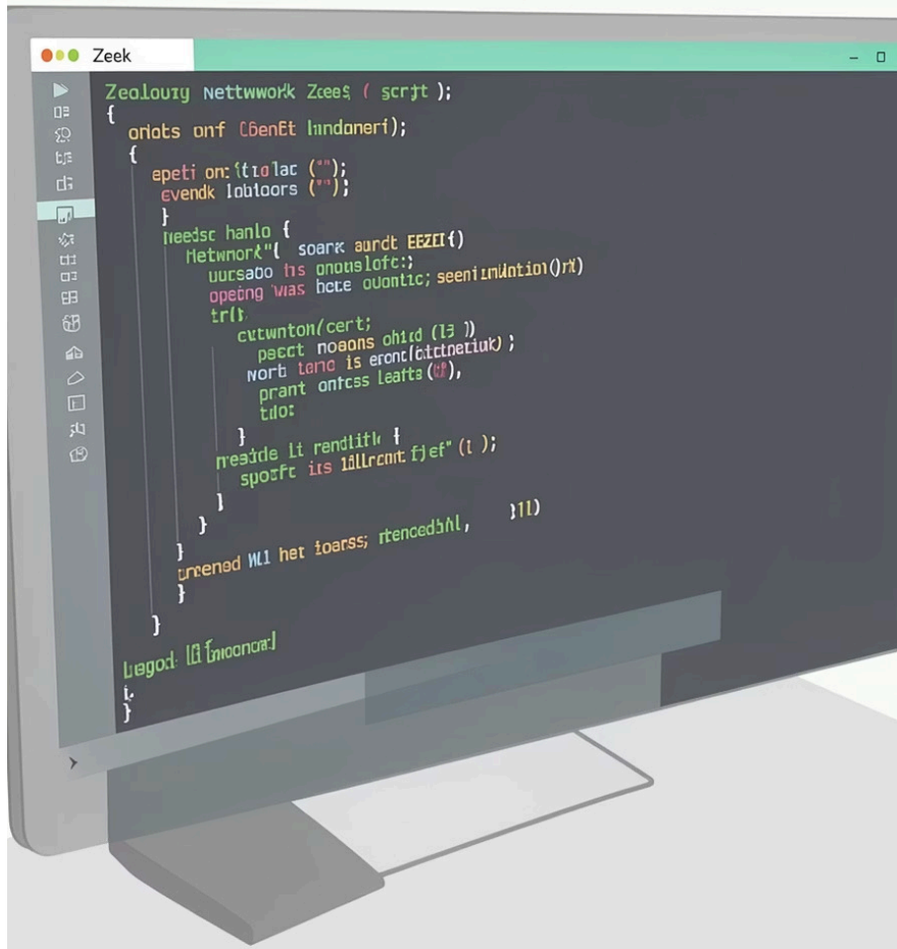
- Using `const` and `set[port]` makes modularity and updating easier.
- It can be loaded or redefined by configuration for different lab or production scenarios.

main.zeek — detection engine

Description and key fragments of the main script that listens for new connections and generates alerts.

```
# Illustrative fragment (main.zeek)
event new_connection(c: connection)
{
    local src = c$id$resp_h;     # Source IP
    local dst_port = c$id$resp_p;# Destination port
    if ( dst_port in suspicious_ports )
        NOTICE([$note="SuspiciousPort", $msg=fmt("Connection to %s
from %s", dst_port, src)]);
}
```

- **Event** `new_connection`: triggered when Zeek detects a new TCP connection.
- Extraction of fields from `connection`: source IP (`resp_h`) and destination port (`resp_p`).
- Evaluation against the blacklist and generation of `NOTICE` for alerts on screen and in notice.log.



attack.py — attack simulator

Goal: generate malicious/controlled traffic that triggers the Zeek rule.

```
# Illustrative snippet (attack.py)
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(1)
try:
    s.connect(("127.0.0.1", 6666))
except Exception:
    pass
s.close()
```

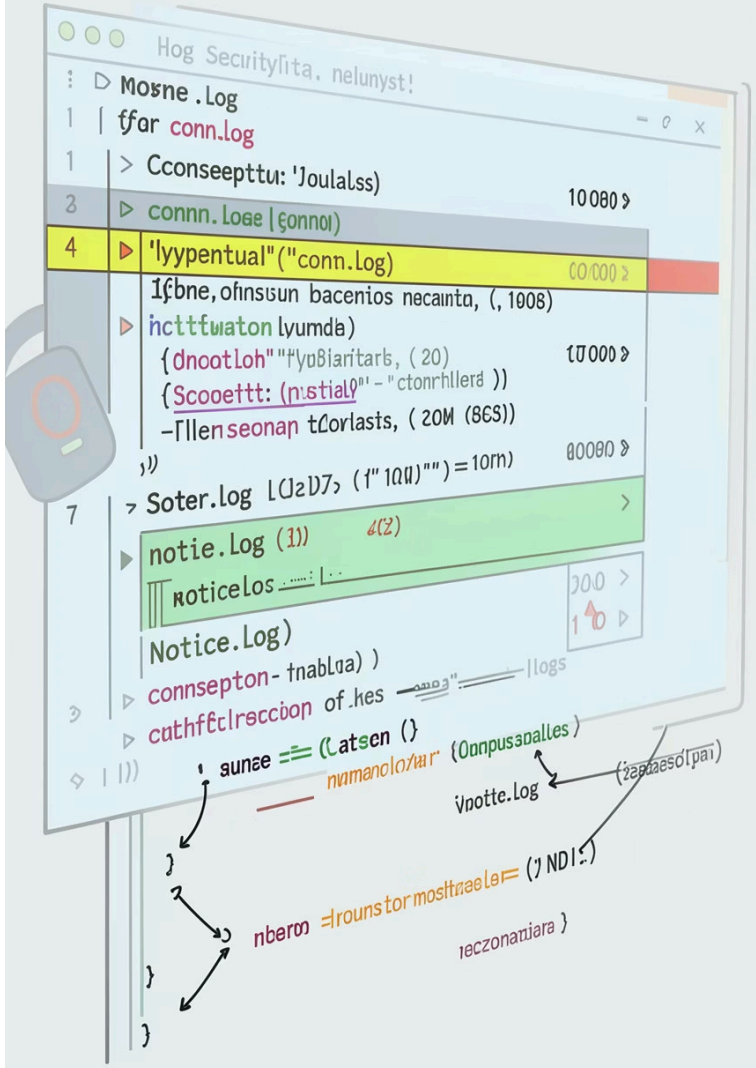
- Uses the standard `socket` library to open TCP connection attempts (SYN).
- Allows repeatable tests, port variation, and traffic generation for detection validation.
- Can be extended to send payloads or more complex simulations.

Zeek logs

- **conn.log**: structured record of connections (5-tuple, duration, bytes transferred).
- **notice.log**: alerts generated by NOTICE (alerts readable by humans and machines).
- **weird.log**: anomalous events or unusual parsing.
- In professional environments, these files are exported to a SIEM for correlation, alerting, and response.

Brief example (summary format):

```
# notice.log (example)
time uid note msg
... - SuspiciousPort Connection to 6666/tcp from 127.0.0.1
```



Best practices and considerations

Modularity

Separating lists (list.zeek) from the logic (main.zeek) makes maintenance and testing easier.

Centralize logs

Send conn.log and notice.log to a SIEM for correlation and alerts at scale.

Continuous testing

Automate simulated attacks (attack.py) to validate rules and avoid false negatives.

Monitoring environment

Monitor relevant interfaces, adjust sensitivity, and document changes in rules.

Conclusions and next steps

- Zeek offers a powerful and extensible platform for event-based network visibility: ideal for forensic analysis and detection.
- Separating blacklists and the detection engine improves maintainability and reusability across different environments.
- Integration of Zeek with SIEM and automated testing with scripts (ataque.py) are essential steps for operational defense.
- Suggested next steps: expand rules (protocols), enrich logs with metadata, and create an alert dashboard in SIEM.

Thank you — questions

MARCO PINO MATIAS REYES JOAQUIN CARRANZA JUAN ANDINO